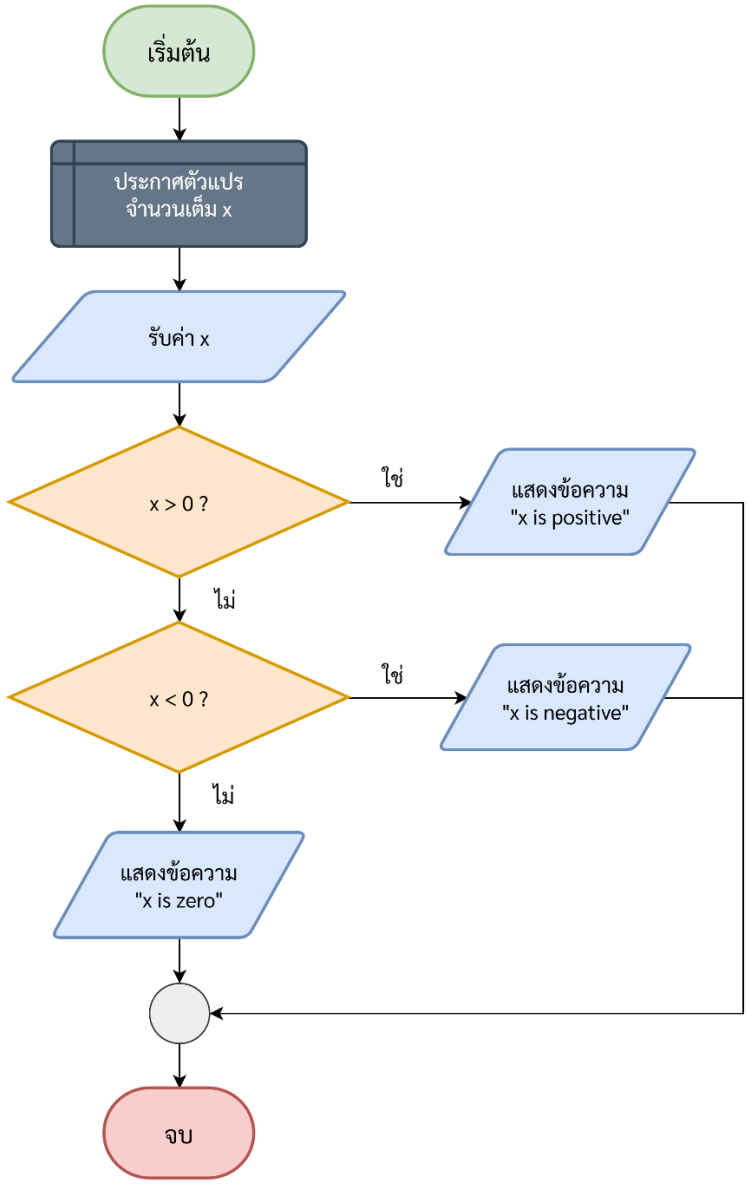
ข้อที่ 1

สำหรับข้อที่ 1 เป็นการศึกษาการใช้ Nested condition statement ดังที่จะเห็นได้จากโจทย์ที่มีการตรวจสอบค่าจำนวนเต็มของตัวแปร x โดยค่า x ที่เป็นจำนวนเต็มบวกจะแสดงข้อความ “x is positive” หากเป็นเท็จจะทำงานต่อไปโดยจะตรวจสอบตัวแปร x หากเป็นจำนวนเต็มลบจะแสดงข้อความ “x is negative” ส่วนถ้าเป็นเท็จจะแสดงข้อความ “x is zero” ดังที่จะได้แสดง ขั้นตอนการทำงานของโปรแกรมโดยใช้ผังงานในลำดับถัดไป

Flowchart แสดงการทำงานของโปรแกรม



Code และตัวอย่าง input/output

```python
'''
6901012610544 นายสุทธิภัทร เอื้อพัฒนากิจ
หัวข้อ Conditional
'''


x = int(input("Enter number for variable x: "))

if (x>0):
    print("x is positive")
else:
    if(x<0):
        print("x is negative")
    else:
        print("x is zero")

```

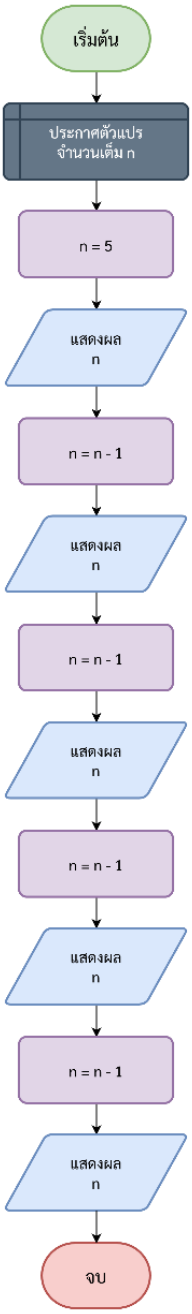
```
Enter number for variable x: 5
x is positive
Enter number for variable x: -8
x is negative
Enter number for variable x: 0
x is zero
```

ข้อที่ 2

ในข้อนี้จะเป็นการศึกษาเกี่ยวกับการกำหนดค่าตัวแปรตามลำดับขั้นตอนโดยเริ่มจากบรรทัดบนสุดคือตัวแปรจำนวนเต็ม $n = 5$ และแสดงผลตัวแปร n จากนั้นบรรทัดต่อมาทำการลดค่า n ไป 1 หน่วย ปล้วแสดงผลตัสแปร n และทำจนถึงบรรทัดล่างสุดที่ $n = 1$

กล่าวคือ โจทย์ต้องการแสดงให้เห็นการทำงานของหน่วยความจำว่า ตัวแปร n สามารถถูกเขียนทับและเปลี่ยนค่าไปได้เรื่อยๆ ส่งผลให้คำสั่ง print(n) ในแต่ละบรรทัดให้ผลลัพธ์ที่ไม่เหมือนกัน ซึ่งจะปูทางไปสู่การเรียนเรื่องการวนซ้ำทั้งในรูปแบบของ While loop และ For loop

Flowchart แสดงการทำงานของโปรแกรม



Code และตัวอย่าง output

```
'''
6901012610544 นายสุทธิภัทร เอื้อพัฒนากิจ
หัวข้อ Variable
'''

n = 5
print(n)

n = n - 1
print(n)

n = n - 1
print(n)

n = n - 1
print(n)

n = n - 1
print(n)
```

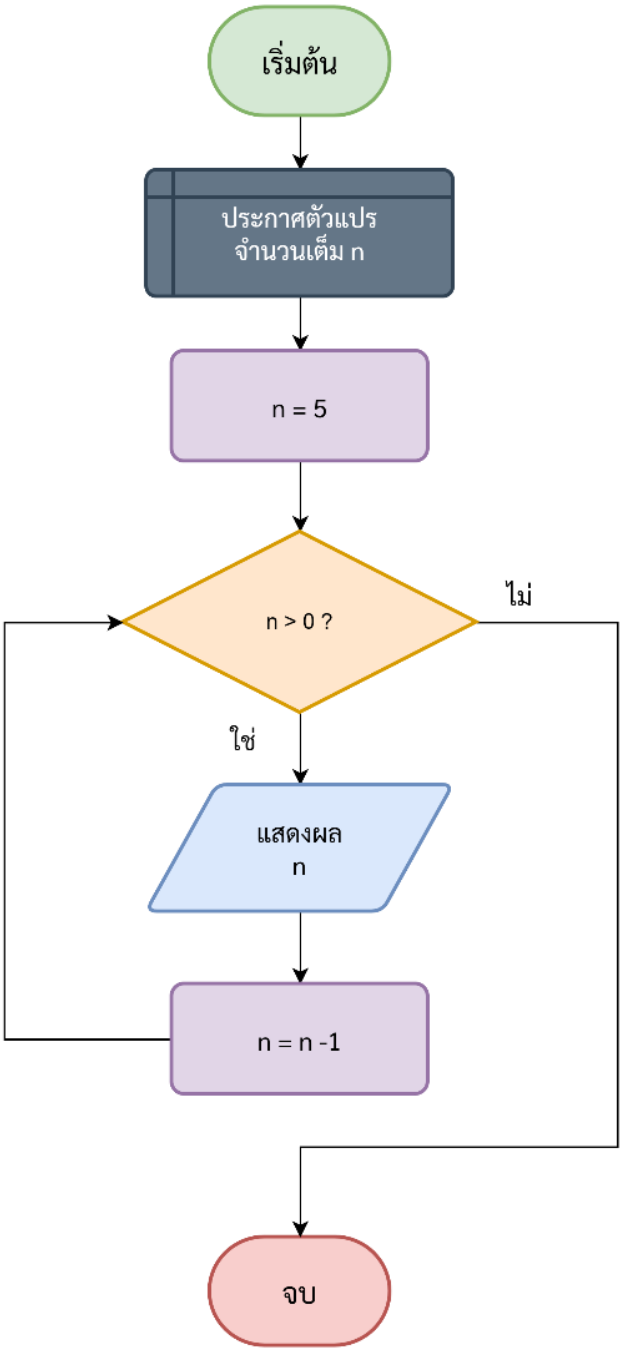
```
5
4
3
2
1
PS C:\Users\Suttipat\AppData\Local\Programs\Microsoft VS Code>
```

ข้อที่ 3

จากข้อก่อนหน้าที่ใช้การลดค่า n ซ้ำๆ เราสามารถลดรูปและเขียนโดยใช้ While loop ได้ในภาษา Python การใช้งาน While loop จะเกิดขึ้นเมื่อรู้เพียงเงื่อนไขบางอย่างที่ทำให้เกิดการวนรอบ โดยลูปแบบ While จะมีการตรวจสอบเงื่อนไขก่อวนรอบ หากตรงตามที่กำหนดก็จะเริ่มคำสั่งในลูป แล้ววนรอบไปเรื่อยๆ จนกว่าเงื่อนไขจะเป็นเท็จ

จากแบบฝึกลูป while จะอสดงค่า n ก็ต่อเมื่อ $n > 0$ โดยเริ่มต้นกำหนดให้ $n = 5$ แล้วตรวจสอบเงื่อนไขของลูป หากเป็นจริงจะแสดงค่า n แล้วลดค่า n ไป 1 หน่วย ทำซ้ำจนเงื่อนไขนั้นเป็นเท็จ

Flowchart แสดงการทำงานของโปรแกรม



Code และตัวอย่าง output

```
1  '''
2  6901012610544 นายสุทธิภัทร เอื้อพัฒนากิจ
3  หัวข้อ While statement
4  '''
5
6  n = 5
7
8  while n > 0:
9      print(n)
10     n = n - 1
```

```
PS C:\Users\Suttipat\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Suttipat\AppData\Local\Python\p
ythoncore-3.14-64\python.exe d:/KMUTNB/1-69/Computer-Programming/week4/Homework/HW/6901012610544-HW3.py
5
4
3
2
1
PS C:\Users\Suttipat\AppData\Local\Programs\Microsoft VS Code>
```

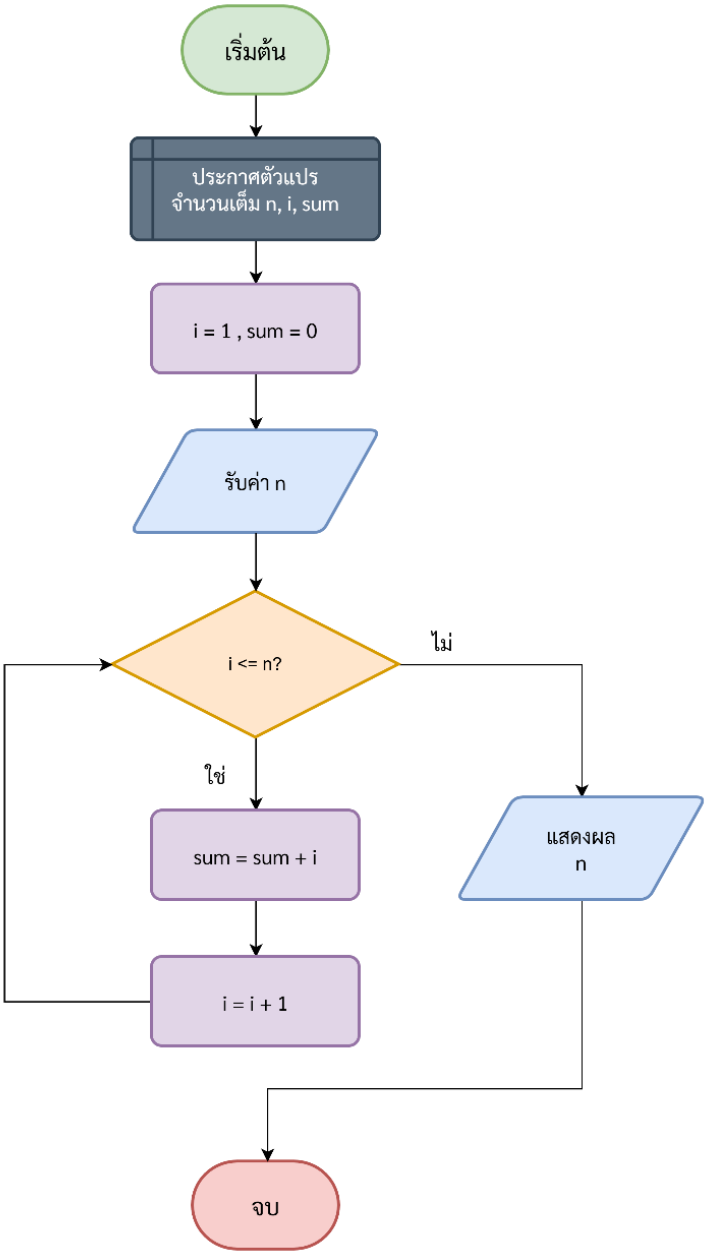
ข้อที่ 4

ก่อนหน้านี้เราแค่วนลูปเพื่อพิมพ์ค่าออกหน้าจอแล้วก็จบไป แต่ข้อต้องการสร้างตัวแปรขึ้นมาทำหน้าที่สะสมค่า (ในสไลด์คือการตั้งค่าเริ่มต้น sum = 0 ไว้ก่อนเริ่มลูป) เพื่อเตรียมรับค่าที่จะโยนเข้ามาบวกทบไปเรื่อยๆ ในแต่ละรอบ

อีกจุดประสงค์คือการฝึกควบคุมตัวแปร 2 ตัวที่ทำงานสัมพันธ์กัน โจทย์ข้อนี้แตกต่างจากข้อก่อนๆ เพราะต้องคุมตัวแปรพร้อมกันถึง 2 ตัวในลูปเดียวคือ

- ตัวควบคุมลูป: ตัวแปร i ทำหน้าที่นับเลขขยับไปทีละขั้น ($1 \rightarrow 2 \rightarrow 3 \rightarrow \ldots \rightarrow n$)
- ตัวเก็บผลรวม: ทำหน้าที่คอยเอาเลขที่ได้จากตัวควบคุมลูปมาบวกทบเข้าตัวเองไปเรื่อย ๆ (sum = sum + ตัวเลขนั้น)

Flowchart แสดงการทำงานของโปรแกรม



Code และตัวอย่าง input/output

```python
'''
6901012610544 นายสุทธิภัทร เอื้อพัฒนากิจ
หัวข้อ Sum
'''

i = 1
sum = 0
n = int(input())

while(i <= n):
    print(f"i = {i}")
    sum = sum + i
    print(f"sum {sum}")
    i = i + 1


print(f"Final summation from 1 to {n} is {sum}")
```

```
PS C:\Users\Suttipat\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Suttipat\AppData\Local\Python\p
ythoncore-3.14-64\python.exe d:/KMUTNB/1-69/Computer-Programming/week4/Homework/HW/6901012610544-HW4.py
5
i = 1
sum 1
i = 2
sum 3
i = 3
sum 6
i = 4
sum 10
i = 5
sum 15
Final summation from 1 to 5 is 15
PS C:\Users\Suttipat\AppData\Local\Programs\Microsoft VS Code>
```

ข้อที่ 5

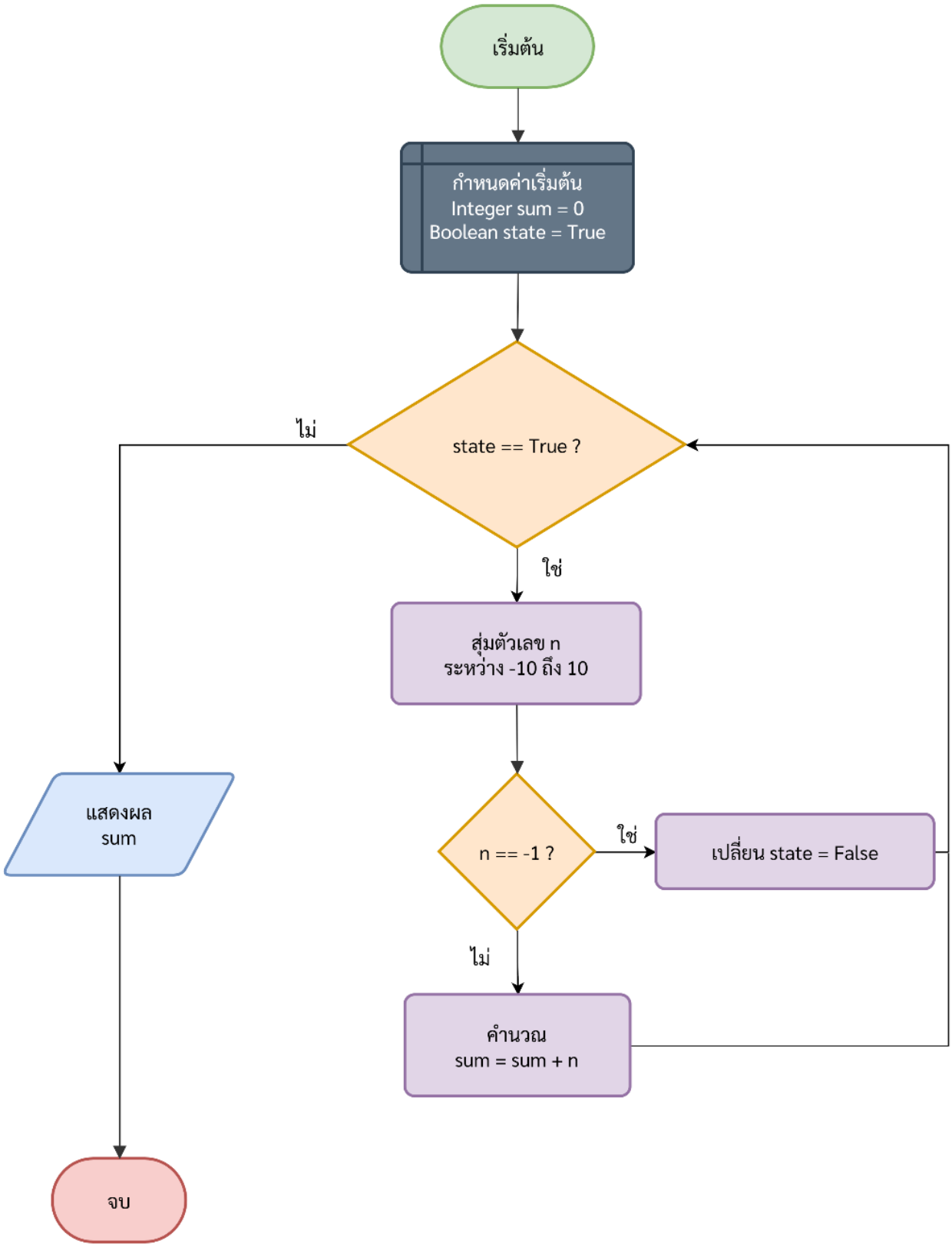
โปรแกรมนี้เริ่มต้นด้วยการดึงโมดูล random มาใช้สำหรับการสุ่มตัวเลข จากนั้นทำการสร้างตัวแปร sum กำหนดค่าเริ่มต้นเป็น 0 เพื่อใช้เก็บผลรวมสะสม และกำหนดตัวแปร state มีค่าเป็น True เพื่อใช้ควบคุมการทำงานของลูปวนซ้ำ while

เมื่อเข้าสู่ลูป while โปรแกรมจะทำการสุ่มจำนวนเต็มในช่วง -10 ถึง 10 ออกมาหนึ่งค่า และเก็บไว้ในตัวแปร n จากนั้นจะตรวจสอบเงื่อนไขด้วยคำสั่ง if-else โดยแบ่งเป็น 2 กรณี

1. กรณีที่สุ่มได้ตัวเลขที่ไม่ใช่ -1: โปรแกรมจะแสดงค่า n ออกทางหน้าจอ นำค่า n ไปบวกสะสมเข้ากับตัวแปร sum และแสดงผลรวมสะสมล่าสุด จากนั้นจะวนกลับไปสุ่มตัวเลขใหม่ในรอบถัดไป
2. กรณีที่สุ่มได้ตัวเลข -1: โปรแกรมจะแสดงค่า n และแสดงข้อความแจ้งว่าพบเลข -1 ซึ่งเป็นสัญญาณสิ้นสุดการบวกสะสม จากนั้นจะปรับค่า state ให้เป็น False ทำให้เงื่อนไขของลูป while ไม่เป็นจริงอีกต่อไป และหลุดออกจากลูปการทำงาน

เมื่อออกจากลูปเรียบร้อยแล้ว โปรแกรมจะทำคำสั่งสุดท้ายโดยการแสดงผลรวมสะสมทั้งหมดที่ได้จากการสุ่มตัวเลขออกทางหน้าจอ ถือเป็นการสิ้นสุดการทำงานของโปรแกรม

Flowchart แสดงการทำงานของโปรแกรม



Code และตัวอย่าง output

```python
'''
หัวข้อ Loop until
'''

import random

sum = 0
state = True

while state:
    n = random.randint(-10, 10)
    if n == -1:
        print(f"n = {n}")
        print("-1 has entered, end of summation")
        state = False
    else:
        print(f"n = {n}")
        sum = sum + n
        print(f"sum = {sum}")

print(f"Therefore, the summation of random sequences is {sum}")
```

```
PS C:\Users\Suttipat\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Suttipat\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/KMUTNB/1-69/Computer-Programming/week4/Homework/HW/6901012610544-HW5.py
n = -6
sum = -6
n = -5
sum = -11
n = 5
sum = -6
n = -9
sum = -15
n = 7
sum = -8
n = -1
-1 has entered, end of summation
Therefore, the summation of random sequences is -8
PS C:\Users\Suttipat\AppData\Local\Programs\Microsoft VS Code>
```

ข้อที่ 6

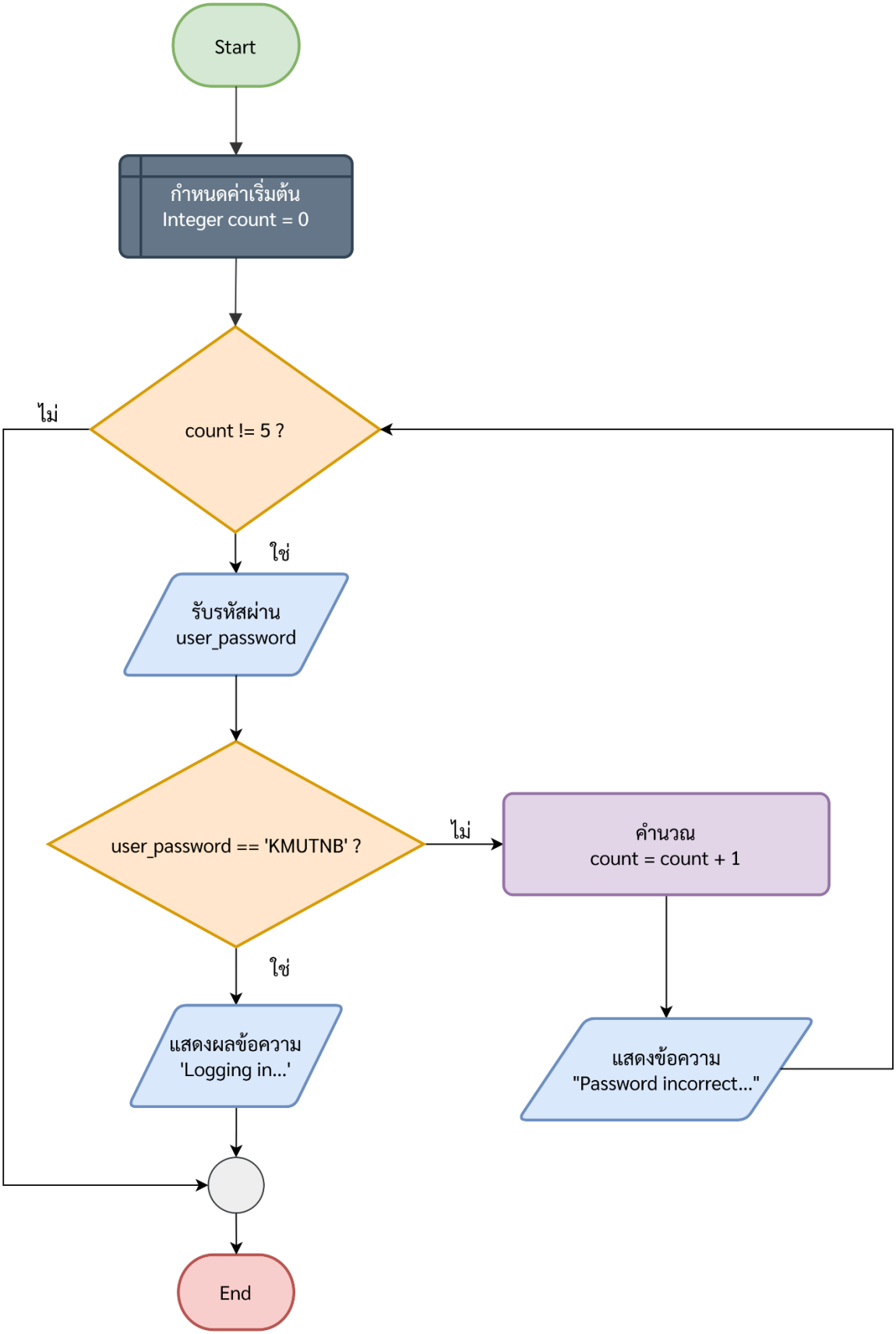
โปรแกรมนี้เริ่มต้นด้วยการนำเข้าโมดูล getpass มาใช้สำหรับรับค่ารหัสผ่านแบบซ่อนข้อความขณะพิมพ์เพื่อความปลอดภัย จากนั้นทำการกำหนดตัวแปร count ให้มีค่าเริ่มต้นเป็น 0 เพื่อใช้นับจำนวนครั้งที่ผู้ใช้ป้อนรหัสผ่านผิด และสร้างการทำงานแบบวนซ้ำด้วยลูป while โดยมีเงื่อนไขให้ทำงานตราบใดที่ count ยังมีค่าไม่เท่ากับ 5

ในแต่ละรอบของการวนลูป โปรแกรมจะรับค่ารหัสผ่านจากผู้ใช้ผ่านฟังก์ชัน getpass.getpass() และนำมาเก็บไว้ในตัวแปร user_password จากนั้นจะทำการตรวจสอบเงื่อนไขด้วยคำสั่ง if-else

1. กรณีที่รหัสผ่านถูกต้อง (ตรงกับ "KMUTNB"): โปรแกรมจะแสดงข้อความ "Logging in..." เพื่อแจ้งว่าเข้าสู่ระบบสำเร็จ และใช้คำสั่ง break เพื่อออกจากลูปทันที
2. กรณีที่รหัสผ่านไม่ถูกต้อง: โปรแกรมจะเข้าสู่บล็อก else โดยทำการบวกค่าตัวแปร count เพิ่มขึ้น 1 ค่า แล้วแสดงข้อความแจ้งเตือนว่ารหัสผ่านไม่ถูกต้อง พร้อมแสดงสัดส่วนจำนวนครั้งที่ป้อนผิดไปแล้ว จากนั้นจะวนกลับไปรับรหัสผ่านใหม่อีกครั้ง

หากผู้ใช้ใส่รหัสผ่านผิดติดต่อกันจนครบ 5 ครั้ง ค่าของ count จะกลายเป็น 5 ทำให้เงื่อนไข count != 5 ของลูป while เป็นเท็จ และโปรแกรมจะจบการทำงานทันที

Flowchart แสดงการทำงาน



Code และตัวอย่าง input/output

```python
'''
6901012610544 นายสุทธิภัทร เอื้อพัฒนากิจ
หัวข้อ Password
'''

import getpass

count = 0


while count != 5:
    user_password = getpass.getpass("Enter your password")
    if user_password == "KMUTNB":
        print("Logging in...")
        break
    else:
        count = count + 1
        print(f"Password incorrect. Please try again {count}/5")
```

กรณีกรอกรหัสผ่านผิดครบ 5 ครั้ง

```
PS C:\Users\Suttipat\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Suttipat\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/KMUTNB/1-69/Computer-Programming/week4/Homework/HW/6901012610544-HW6.py
Enter your password
Password incorrect. Please try again 1/5
Enter your password
Password incorrect. Please try again 2/5
Enter your password
Password incorrect. Please try again 3/5
Enter your password
Password incorrect. Please try again 4/5
Enter your password
Password incorrect. Please try again 5/5
PS C:\Users\Suttipat\AppData\Local\Programs\Microsoft VS Code>
```

กรณีกรอกรหัสผ่านถูก คือ KMUTNB

```
PS C:\Users\Suttipat\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Suttipat\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/KMUTNB/1-69/Computer-Programming/week4/Homework/HW/6901012610544-HW6.py
Enter your password
Logging in...
PS C:\Users\Suttipat\AppData\Local\Programs\Microsoft VS Code>
```

ข้อที่ 7

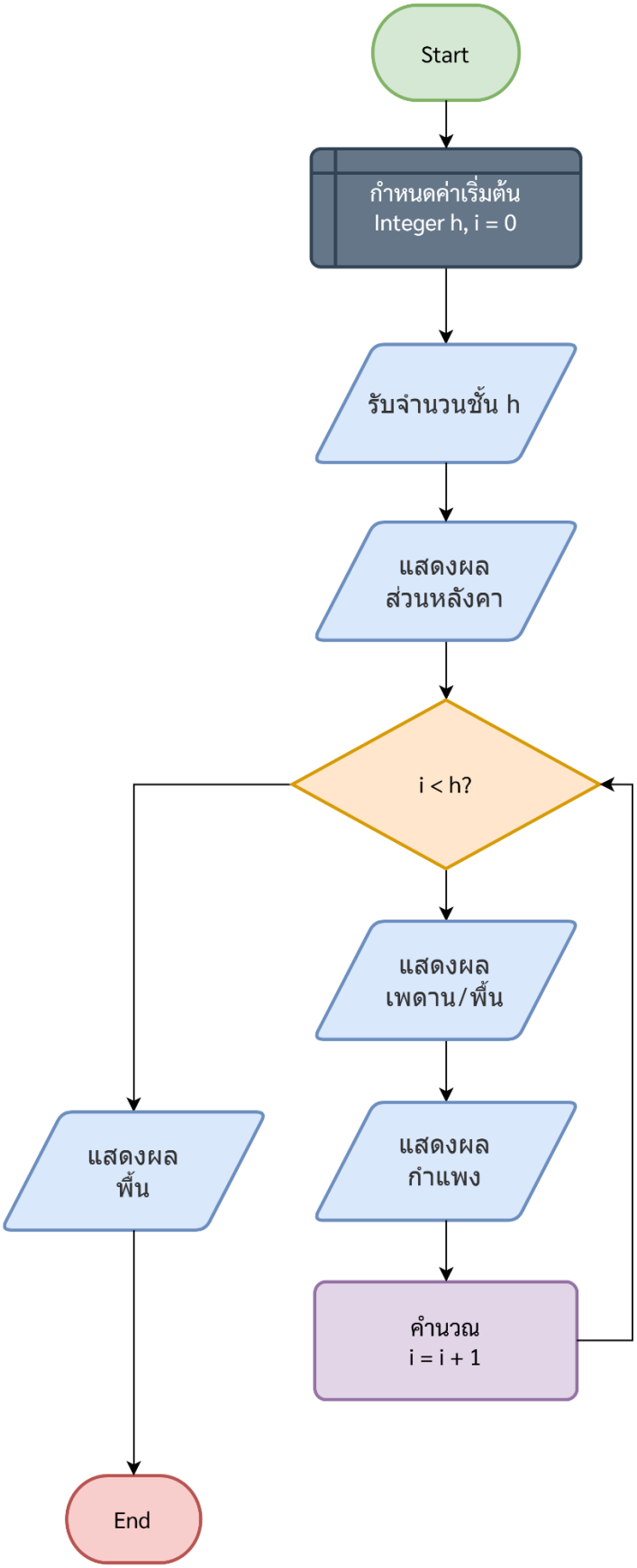
โปรแกรมนี้เริ่มต้นด้วยการรับข้อมูลจำนวนชั้นของอาคารที่ผู้ใช้ต้องการจากแป้นพิมพ์ผ่านคำสั่ง input() และทำการแปลงประเภทข้อมูลจากข้อความให้เป็นจำนวนเต็มด้วยคำสั่ง int() จากนั้นนำไปเก็บไว้ในตัวแปร h

ถัดมา โปรแกรมจะทำการแสดงผลรูปหลังคาของอาคารออกทางหน้าจอเป็นทรงสามเหลี่ยมด้วยคำสั่ง print() โดยใช้สัญลักษณ์ \n เพื่อจัดรูปแบบการขึ้นบรรทัดใหม่

จากนั้น โปรแกรมจะเข้าสู่ขั้นตอนการวาดชั้นอาคารตามจำนวนที่กำหนด โดยใช้การวนลูป for ร่วมกับ range(0, h) เพื่อทำงานซ้ำเท่ากับจำนวนชั้น h รอบ ในแต่ละรอบของการวนลูป โปรแกรมจะพิมพ์ส่วนขอบชั้น (|||||) ตามด้วยผนังอาคารความสูง 2 บรรทัด (| |) ต่อเนื่องกันไปจนครบตามจำนวนชั้นที่ระบุ

เมื่อโปรแกรมวาดชั้นอาคารครบทุกชั้นและออกจากลูปแล้ว จะทำคำสั่งสุดท้าย print("|||||") เพื่อพิมพ์ส่วนฐานปิดท้ายที่ล่างสุดของอาคาร ถือเป็นการสิ้นสุดการทำงานของโปรแกรม

Flowchart แสดงการทำงาน



Code และตัวอย่าง input/output



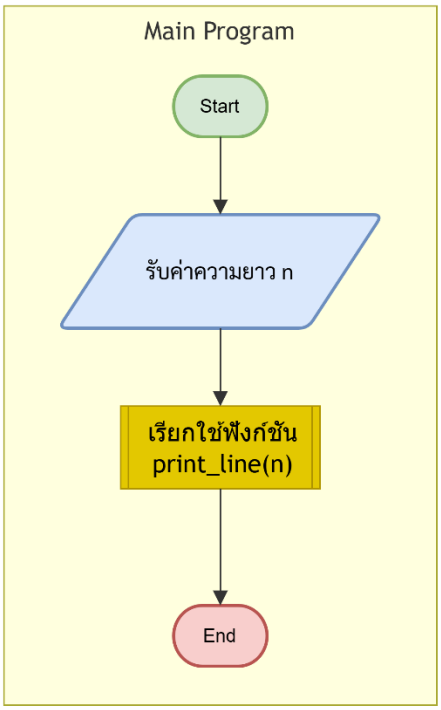
ข้อที่ 8

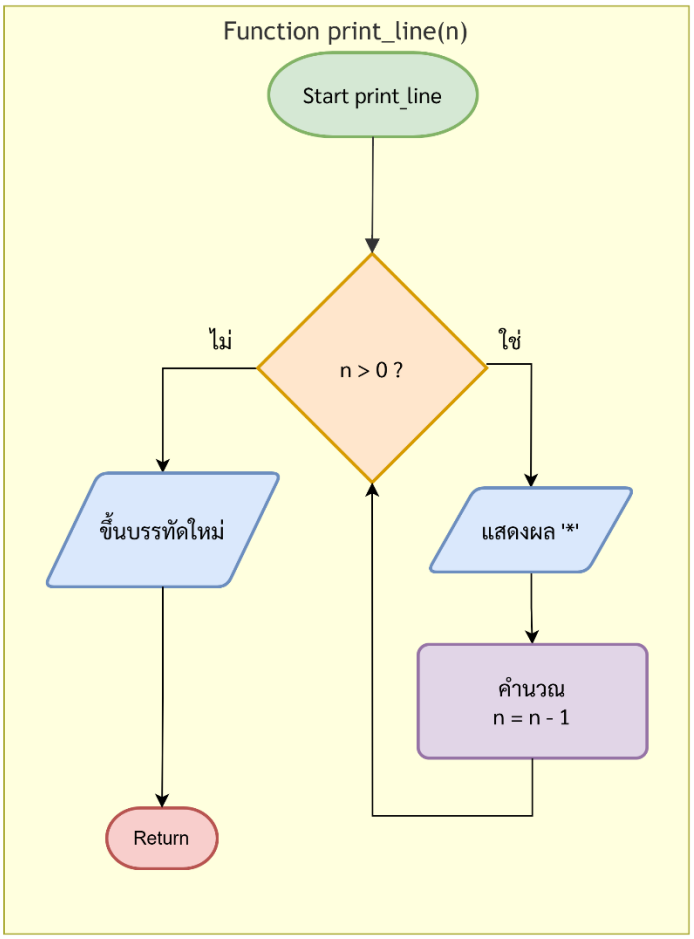
โปรแกรมนี้เริ่มต้นการทำงาน โดยจะรับข้อมูลความยาวของเส้นตรงที่ผู้ใช้ต้องการผ่านแป้นพิมพ์ นำมาแปลงเป็นตัวเลขจำนวนเต็มด้วยคำสั่ง int() และเก็บไว้ในตัวแปร n จากนั้นจะทำการเรียกใช้งานฟังก์ชัน print_line(n) โดยส่งค่าตัวเลข n เข้าไปประมวลผล

เมื่อเข้าสู่การทำงานภายในฟังก์ชัน print_line(n) โปรแกรมจะใช้ลูป while ในการสร้างเส้นตรง โดยมีเงื่อนไขให้ทำงานตราบใดที่ค่า n ยังมากกว่า 0 ในแต่ละรอบของการวนลูป โปรแกรมจะพิมพ์เครื่องหมายดอกจัน (*) ออกทางหน้าจอต่อกันไปเรื่อยๆ ในบรรทัดเดียว โดยใช้ตัวกำหนด end="" และทำการลดค่า n ลงทีละ 1 ค่า

เมื่อพิมพ์เครื่องหมายดอกจันจนครบตามจำนวนความยาวที่ระบุแล้ว ค่า n จะลดลงจนกลายเป็น 0 ทำให้เงื่อนไขของลูป while เป็นเท็จและหลุดออกจากลูป จากนั้นโปรแกรมจะทำคำสั่ง print() เพื่อทำการขึ้นบรรทัดใหม่ ถือเป็นการสิ้นสุดการทำงานของฟังก์ชันและจบการทำงานของโปรแกรม

Flowchart แสดงการทำงาน





Code และตัวอย่าง input/output

Homework > HW > 6901012610544-HW8.py > ...

```python
'''
6901012610544 นายสุทธิภัทร เอื้อพัฒนากิจ
หัวข้อ print_line()
'''

def print_line(n):
    while n > 0:
        print("*", end="")
        n = n - 1
    print()

if __name__ == '__main__':
    n = int(input("How long do you want for your line: "))
    print_line(n)
```

```
How long do you want for your line: 5
*****
How long do you want for your line: 0

How long do you want for your line: 13
*************
```

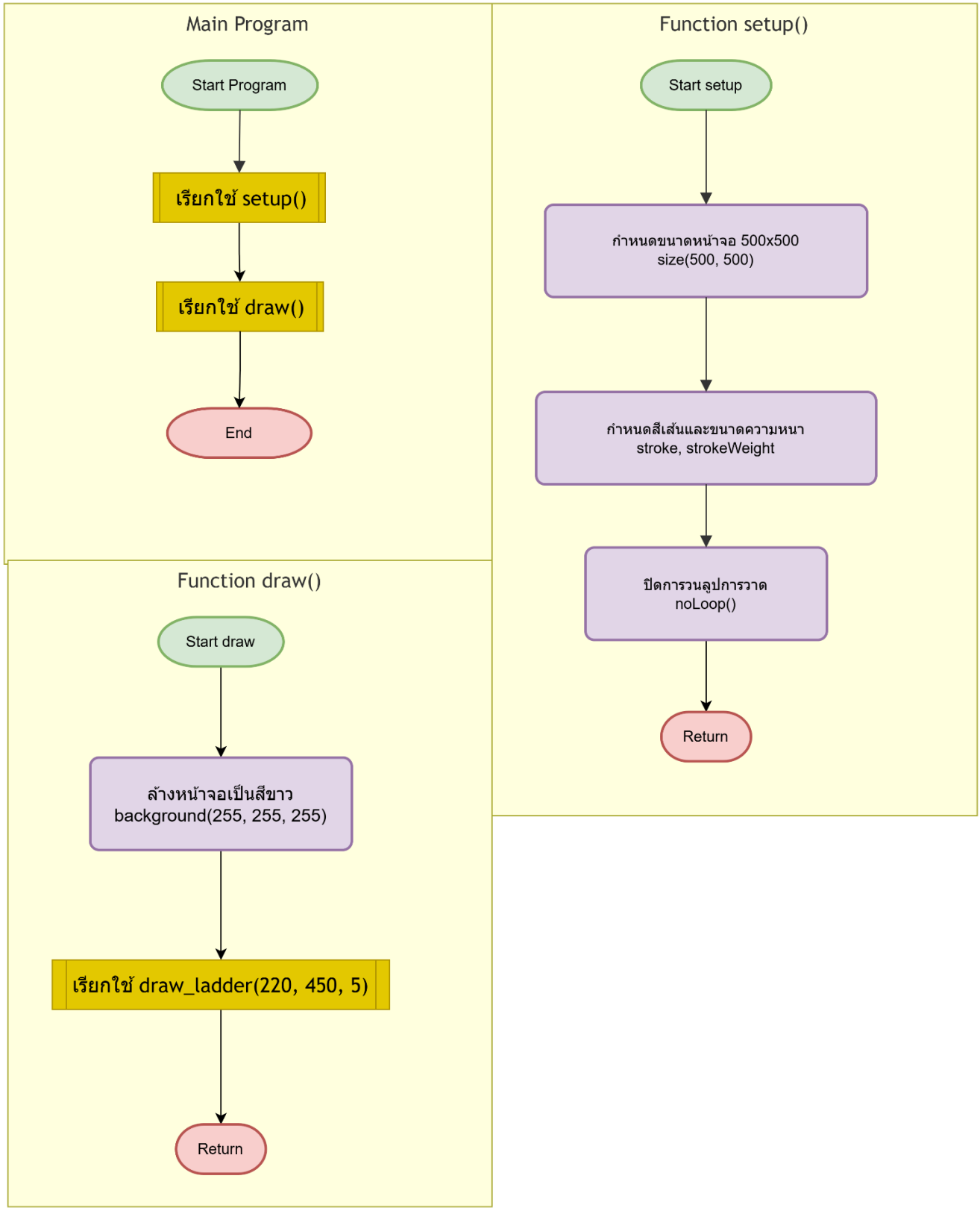
ข้อที่ 9

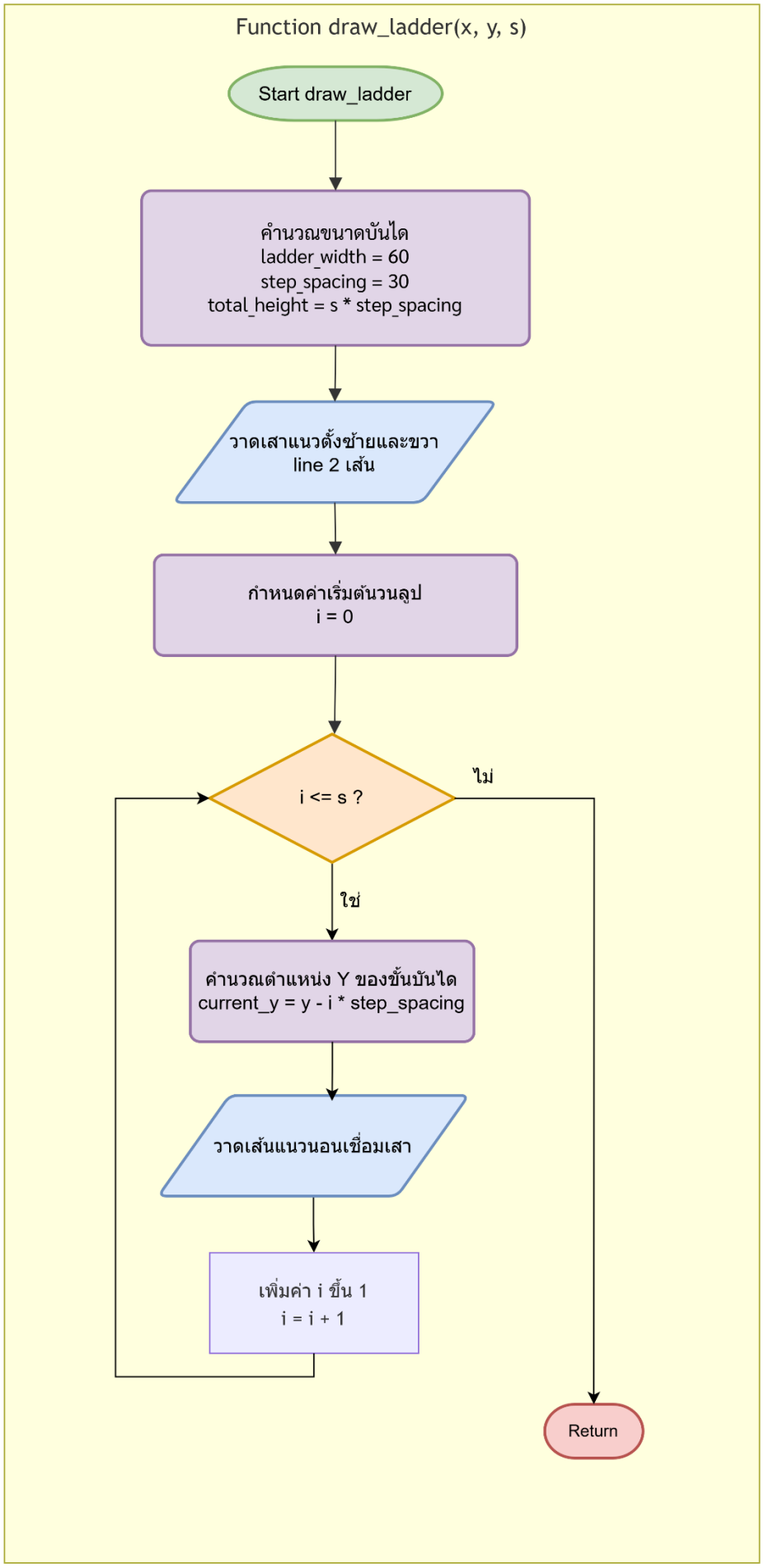
ฟังก์ชัน setup() การทำงานภายในฟังก์ชันจะเริ่มต้นด้วยการสร้างพื้นที่วาดภาพ ขนาดความกว้าง 500 พิกเซล และความสูง 500 พิกเซล ผ่านคำสั่ง size(500, 500) จากนั้นกำหนดสีของเส้นวาดขอบให้เป็นสีดำด้วยคำสั่ง stroke(0) และปรับความหนาของเส้นวาดให้มีขนาด 2 พิกเซลด้วยคำสั่ง strokeWeight(2) ปิดท้ายด้วยคำสั่ง noLoop() เพื่อปิดการวนลูปของระบบวาดภาพ ส่งผลให้โปรแกรมแสดงผลภาพนิ่งเพียงครั้งเดียวโดยไม่ทำการวาดซ้ำ

ฟังก์ชัน draw() เป็นฟังก์ชันที่ทำหน้าที่ควบคุมการแสดงผลกราฟิกบนหน้าจอ โดยในโปรแกรมนี้เริ่มต้นด้วยการล้างและเทสีพื้นหลังของพื้นที่วาดภาพให้เป็นสีขาวด้วยคำสั่ง background(255, 255, 255) จากนั้นทำการเรียกใช้งานฟังก์ชันสร้างรูปภาพเฉพาะอย่าง draw_ladder(x, y, s) โดยการส่งค่าพารามิเตอร์ 3 ค่าเข้าไปประมวลผล ได้แก่ กำหนดให้จุดอ้างอิงฐานบันไดอยู่ที่พิกัด X, พิกัด Y และกำหนดจำนวนขั้นบันไดที่ต้องการวาดไว้ที่ s ขั้น เพื่อทำการวาดรูปบันไดลงบนหน้าจอตามตำแหน่งและขนาดที่ระบุ

ฟังก์ชัน draw_ladder(x, y, s) เป็นฟังก์ชันที่สร้างขึ้นมาเฉพาะเพื่อใช้วาดรูปบันได โดยรับพารามิเตอร์เข้ามา 3 ค่า คือ พิกัด x และ y สำหรับกำหนดตำแหน่งฐานล่างซ้ายของบันได และค่า s สำหรับกำหนดจำนวนขั้นบันไดการทำงานภายในฟังก์ชันเริ่มต้นด้วยการกำหนดค่าคงที่ของบันได ได้แก่ ความกว้างบันไดเท่ากับ 60 พิกเซล (ladder_width) และระยะห่างระหว่างขั้นบันไดเท่ากับ 30 พิกเซล (step_spacing) จากนั้นนำจำนวนขั้น s มาคูณกับระยะห่างเพื่อหาความสูงรวมทั้งหมด แล้วเริ่มวาดเสาแนวตั้งของบันไดทั้งฝั่งซ้ายและฝั่งขวาผ่านคำสั่ง line() ถัดมาจะเข้าสู่กระบวนการวาดขั้นบันไดแนวแนวนอน โดยใช้ลูป while วนซ้ำตั้งแต่ขั้นล่างสุด ($i = 0$) ขึ้นไปจนถึงขั้นบนสุด ($i = s$) ในแต่ละรอบจะคำนวณตำแหน่งพิกัด Y ของขั้นนั้นๆ (นำค่า Y เดิมมาลบด้วยระยะห่างของขั้น) แล้วใช้คำสั่ง line() วาดเส้นแนวนอนเชื่อมระหว่างเสาฝั่งซ้ายและเสาฝั่งขวา เมื่อวาดขั้นบันไดครบตามจำนวน s ขั้นแล้ว โปรแกรมจะออกจากลูปและจบการทำงานของฟังก์ชัน

Flowchart แสดงการทำงาน




Function draw_ladder(x, y, s)
Start draw_ladder
คำนวณขนาดบันได
ladder_width = 60
step_spacing = 30
total_height = s * step_spacing
วาดเสาแนวตั้งซ้ายและขวา
line 2 เส้น
กำหนดค่าเริ่มต้นวนลูป
i = 0
i <= s ?
ไม่
ใช่
คำนวณตำแหน่ง Y ของขั้นบันได
current_y = y - i * step_spacing
วาดเส้นแนวนอนเชื่อมเสา
เพิ่มค่า i ขึ้น 1
i = i + 1
Return

Code และตัวอย่าง input/output

```
def draw():
    background(255,255,255)
    draw_ladder(220, 450, 12)

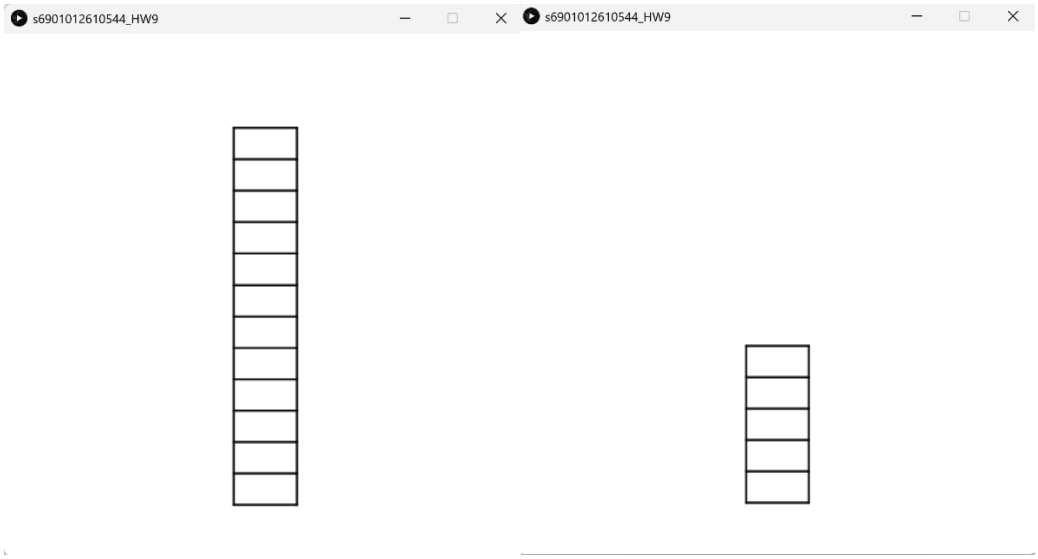
def draw_ladder(x, y, s):
    ladder_width = 60
    step_spacing = 30
    total_height = s * step_spacing

    line(x, y, x, y - total_height)
    line(x + ladder_width, y, x + ladder_width, y - total_height)

    i = 0
    while i <= s:
        current_y = y - (i * step_spacing)
        line(x, current_y, x + ladder_width, current_y)

        i = i + 1
```

การใช้งาน draw_ladder(x, y, s) เมื่อกำหนด s = 12 และ 5 ตามลำดับ



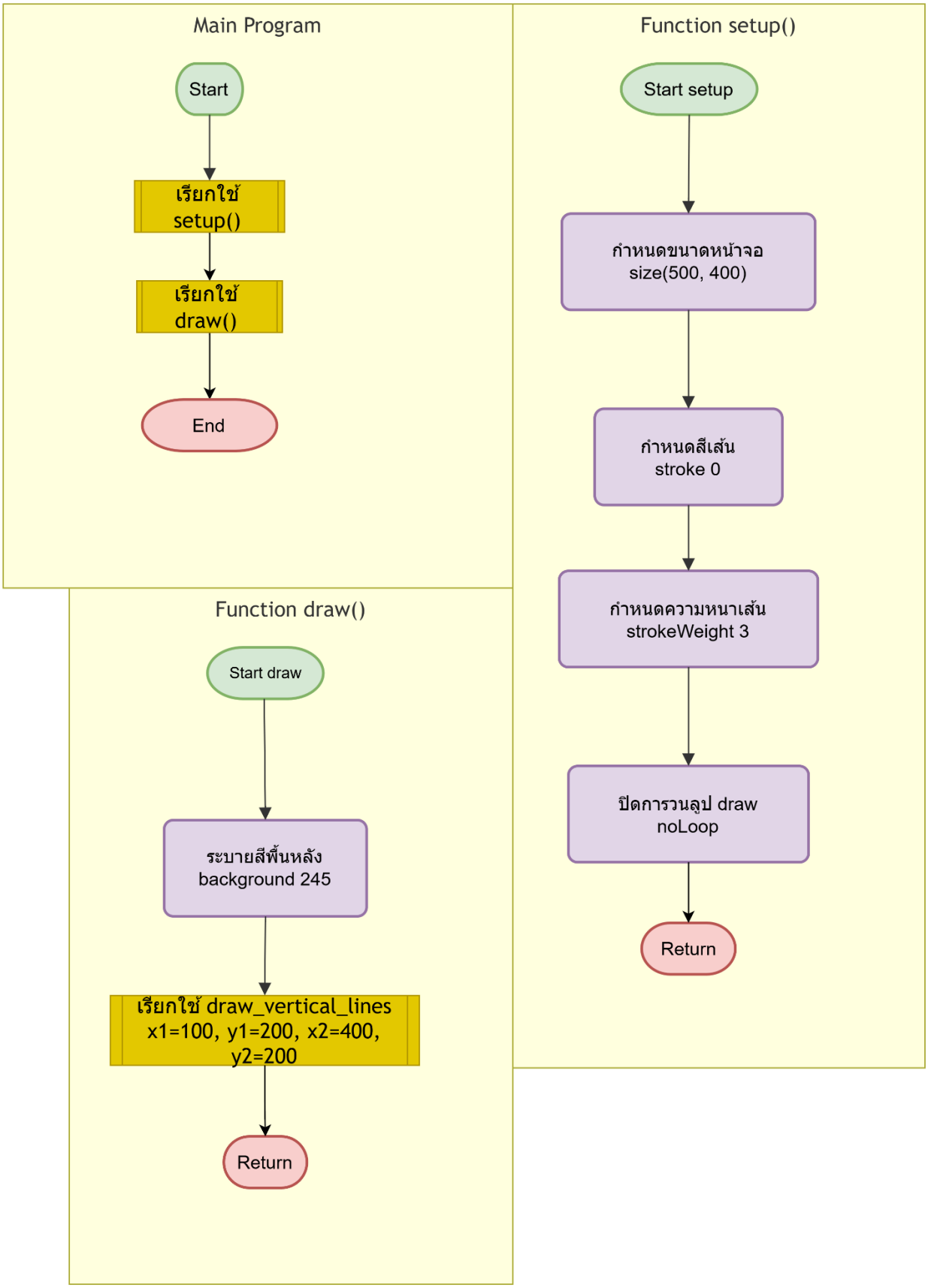
ข้อที่ 10

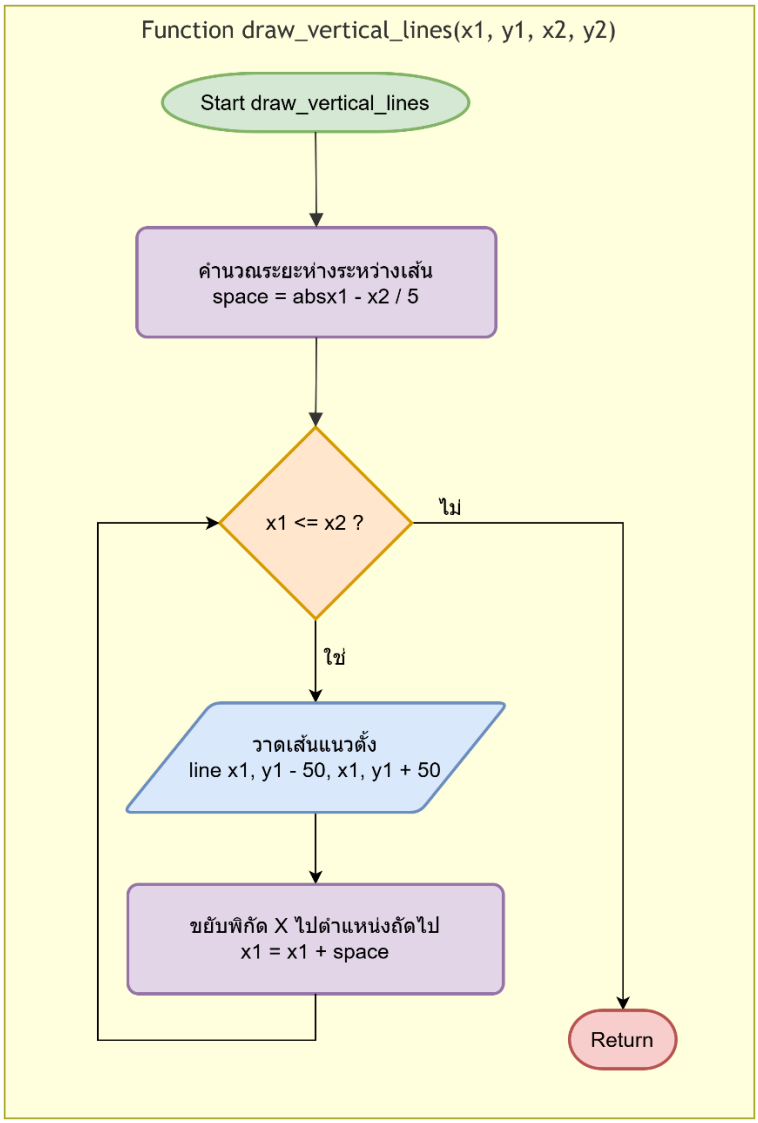
ฟังก์ชัน setup() การทำงานภายในฟังก์ชันเริ่มต้นด้วยการสร้างพื้นที่วาดภาพ กำหนดขนาดความกว้าง 500 พิกเซล และความสูง 400 พิกเซล ด้วยคำสั่ง size(500, 400) จากนั้นกำหนดสีของเส้นวาดให้เป็นสีดำด้วยคำสั่ง stroke(0) และปรับความหนาของเส้นวาดให้มีขนาด 3 พิกเซลผ่านคำสั่ง strokeWeight(3) ปิดท้ายด้วยคำสั่ง noLoop() เพื่อสั่งให้โปรแกรมวาดภาพเพียงครั้งเดียวโดยไม่ต้องวนลูปทำงานซ้ำ

ฟังก์ชัน draw()เริ่มต้นด้วยการเทสีพื้นหลังของ Canvas ให้เป็นสีเทาอ่อนด้วยคำสั่ง background(245) จากนั้นทำการเรียกใช้งานฟังก์ชัน draw_vertical_lines(100, 200, 200, 200) โดยส่งค่าพารามิเตอร์เข้าไปประมวลผล เพื่อระบุตำแหน่งจุดเริ่มต้นและจุดสิ้นสุดของการวาดเส้นแนวตั้งบนหน้าจอที่ขอบเขตพิกัด X ตั้งแต่ 100 ถึง 200 และระบุระดับพิกัด Y อ้างอิงอยู่ที่ตำแหน่ง 200

ฟังก์ชัน draw_vertical_lines(x1, y1, x2, y2) เป็นฟังก์ชันที่สร้างขึ้นเฉพาะเพื่อใช้วาดกลุ่มเส้นตรงแนวตั้งเรียงต่อกันเป็นช่วงขอบเขต โดยรับพารามิเตอร์เข้ามา 4 ค่า คือ พิกัด $(x1, y1)$ สำหรับจุดเริ่มต้น และ $(x2, y2)$ สำหรับจุดสิ้นสุด การทำงานภายในฟังก์ชันเริ่มต้นด้วยการคำนวณระยะห่างระหว่างเส้น (space) โดยนำผลต่างของพิกัด X ทั้งสองฝั่งมาหาค่าสัมบูรณ์แล้วหารด้วย 5 จากนั้นจะใช้ลูป while ในการวนซ้ำวาดเส้นแนวตั้งตราบใดที่ค่า $x1$ ยังน้อยกว่าหรือเท่ากับ $x2$ ในแต่ละรอบจะใช้วาดเส้นตรงแนวตั้งด้วยคำสั่ง line(x1, y1 - 50, x1, y1 + 50) ซึ่งมีความยาวรวม 100 พิกเซล ยืดขึ้นด้านบน 50 พิกเซล และยืดลงด้านล่าง 50 พิกเซล จากพิกัด $y1$ แล้วทำการขยับตำแหน่งพิกัด $x1$ ไปทางขวาด้วยการบวกเพิ่มเท่ากับค่า space จนกระทั่งวาดเส้นขนานแนวตั้งครบตามขอบเขตพื้นที่แล้วจึงออกจากลูป

Flowchart แสดงการทำงาน




Function draw_vertical_lines(x1, y1, x2, y2)
Start draw_vertical_lines
คำนวณระยะห่างระหว่างเส้น
space = absx1 - x2 / 5
x1 <= x2 ?
ไม่
ใช่
วาดเส้นแนวตั้ง
line x1, y1 - 50, x1, y1 + 50
ขยับพิกัด X ไปตำแหน่งถัดไป
x1 = x1 + space
Return

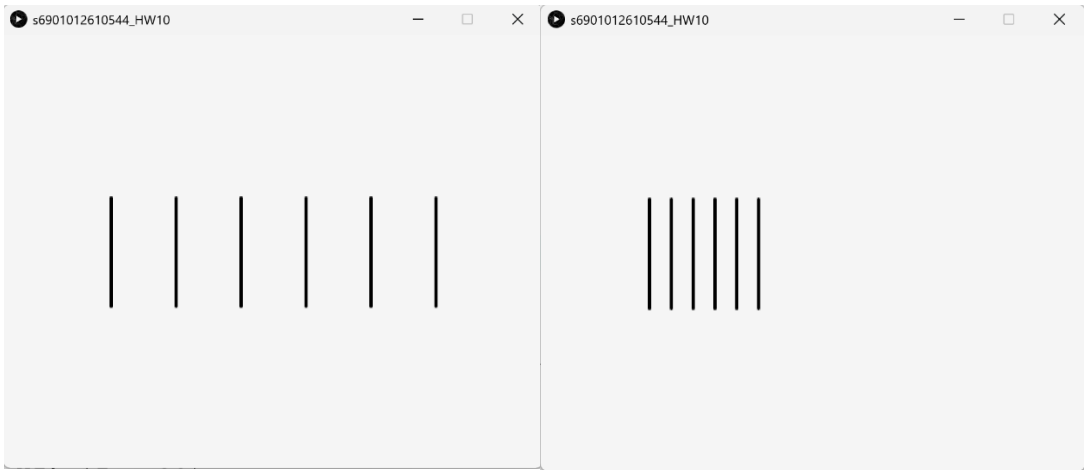
Code และตัวอย่าง Input/output

```
def setup():
    size(500, 400)
    stroke(0)
    strokeWeight(3)
    noLoop()

def draw():
    background(245)
    draw_vertical_lines(100, 200, 400, 200)

def draw_vertical_lines(x1, y1, x2, y2):
    space = abs(x1 - x2)/5

    while x1 <= x2:
        line(x1, y1 - 50, x1, y1 + 50)
        x1 = x1 + space
```



ข้อที่ 11

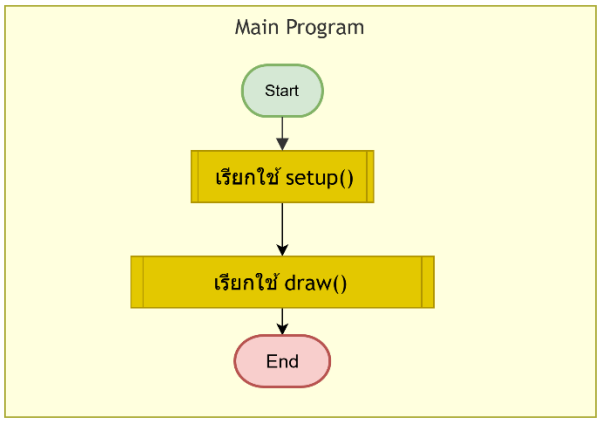
ฟังก์ชัน draw_circle(x, y, r) เป็นฟังก์ชันผู้ช่วย (Helper Function) ที่สร้างขึ้นมาเพื่อวาดรูปวงกลม โดยรับพารามิเตอร์ตำแหน่งจุดศูนย์กลาง x, y และค่ารัศมี r เข้ามาประมวลผล
การทำงานภายในฟังก์ชันจะทำการเรียกใช้คำสั่ง circle(x, y, r * 2) ของ Processing โดยคำนวณนำค่ารัศมี r มาคูณด้วย 2 เพื่อแปลงเป็นค่าความยาวเส้นผ่านศูนย์กลาง (Diameter) ตามที่คำสั่ง circle() ดั้งเดิมต้องการ

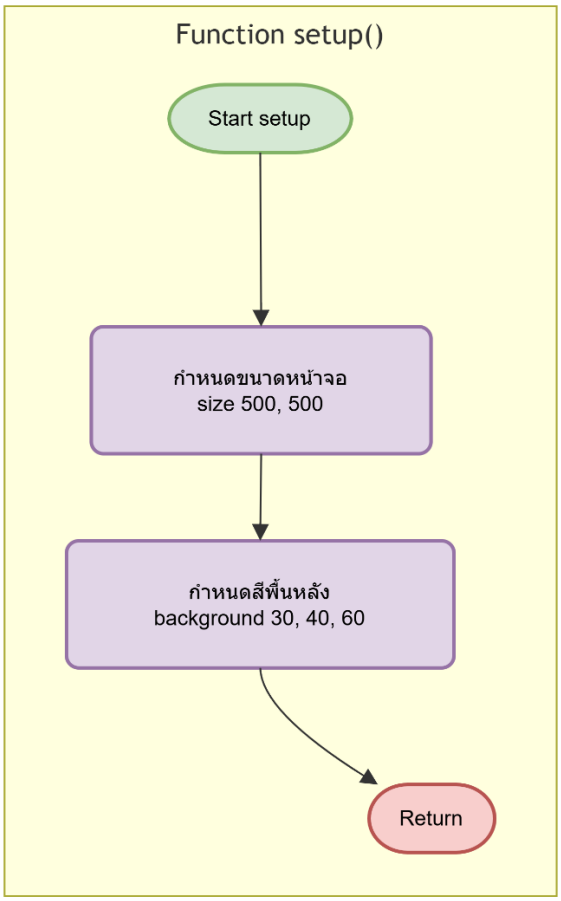
ฟังก์ชัน draw_rain_drops(n) ทำหน้าที่สุ่มวาดรูปวงกลมจำลองหยดน้ำฝนตามจำนวนรอบที่ระบุ โดยรับพารามิเตอร์ n เข้ามาเพื่อกำหนดจำนวนหยดน้ำ
การทำงานภายในฟังก์ชันจะใช้ลูป while ในการวนซ้ำสร้างวงกลม ในแต่ละรอบจะทำการสุ่มพิกัดตำแหน่ง x และ y บนหน้าจอในช่วง 0 ถึง 500 พิกเซล พร้อมกับสุ่มขนาดรัศมี r ในช่วง 3 ถึง 30 พิกเซล แล้วส่งค่านั้นไปวาดรูปวงกลมผ่านฟังก์ชัน draw_circle(x, y, r) ลูปจะวนซ้ำและบวกค่าตัวนับ i เพิ่มขึ้นทีละ 1 จนกระทั่งวาดวงกลมได้ครบตามจำนวน n วงที่กำหนดไว้แล้วจึงออกจากลูป

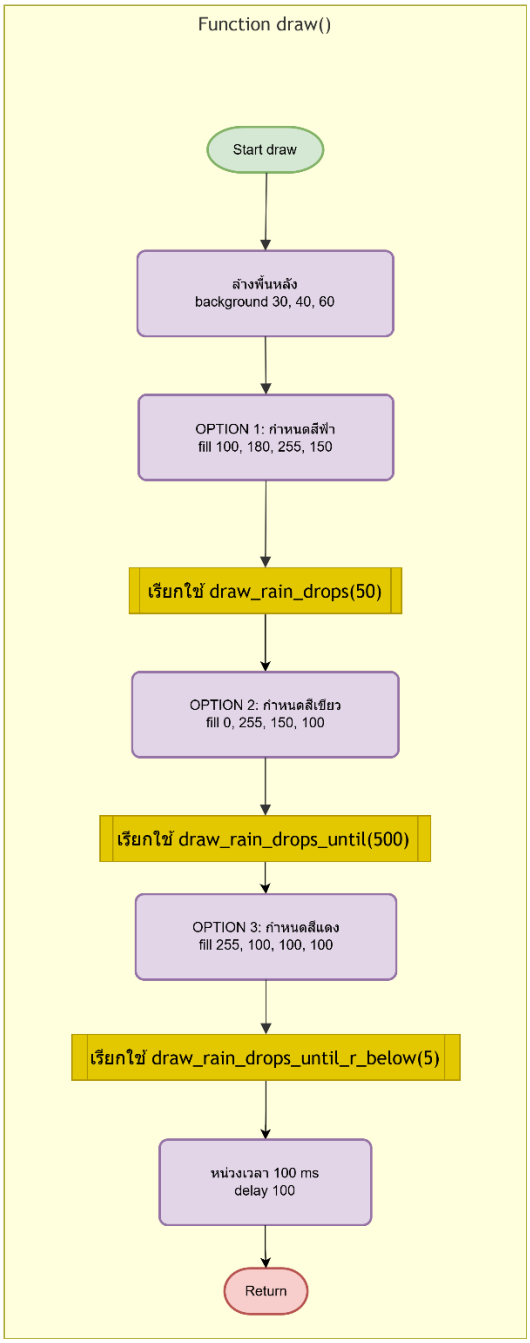
ฟังก์ชัน draw_rain_drops_until(volume) ทำหน้าที่สุ่มวาดรูปวงกลมหยดน้ำสะสมไปเรื่อยๆ จนกว่าผลรวมของรัศมีวงกลมทั้งหมดจะมีค่าถึงหรือเกินกว่าเป้าหมาย volume ที่กำหนดเข้ามา
การทำงานภายในฟังก์ชันเริ่มต้นด้วยการสร้างตัวแปร sum = 0 เพื่อใช้เก็บผลรวมสะสมของรัศมี จากนั้นจะเข้าสู่ลูป while sum < volume: ในแต่ละรอบโปรแกรมจะสุ่มพิกัด x, y และสุ่มรัศมี r (ช่วง 3 ถึง 30 พิกเซล) เพื่อวาดวงกลม จากนั้นนำค่ารัศมี r ที่สุ่มได้ในรอบนั้นไปบวกสะสมเข้ากับตัวแปร sum โปรแกรมจะทำการสุ่มวาดวงกลมไปเรื่อยๆ จนกระทั่งผลรวม sum มีค่ามากกว่าหรือเท่ากับ volume เงื่อนไขลูปจึงเป็นเท็จและหยุดการวาด

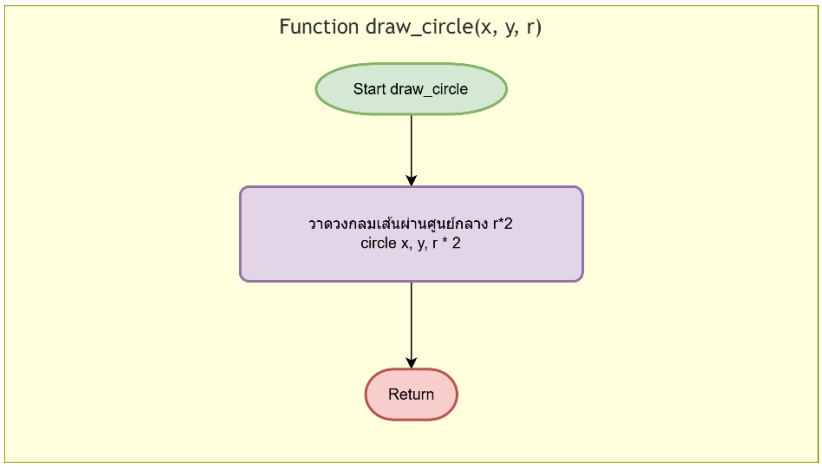
ฟังก์ชัน draw_rain_drops_until_r_below(value) ทำหน้าที่สุ่มวาดรูปวงกลมหยดน้ำไปเรื่อยๆ และจะหยุดวาดเมื่อสุ่มได้วงกลมที่มีขนาดรัศมีเล็กกว่าเงื่อนไข value ที่กำหนดเข้ามา การทำงานภายในฟังก์ชันประยุกต์ใช้เทคนิคตัวแปรสถานะ (Flag Control) โดยกำหนด r_flag = True เพื่อควบคุมการทำงานของลูป while r_flag: ในแต่ละรอบของการวนลูป โปรแกรมจะสุ่มพิกัด x, y และสุ่มรัศมี r เพื่อวาดวงกลม จากนั้นจะตรวจสอบด้วยคำสั่งเงื่อนไข if r < value: หากสุ่มได้รัศมี r ที่มีค่าน้อยกว่า value โปรแกรมจะเปลี่ยนค่า r_flag ให้เป็น False ส่งผลให้การวนลูปสิ้นสุดลงทันทีในรอบถัดไป

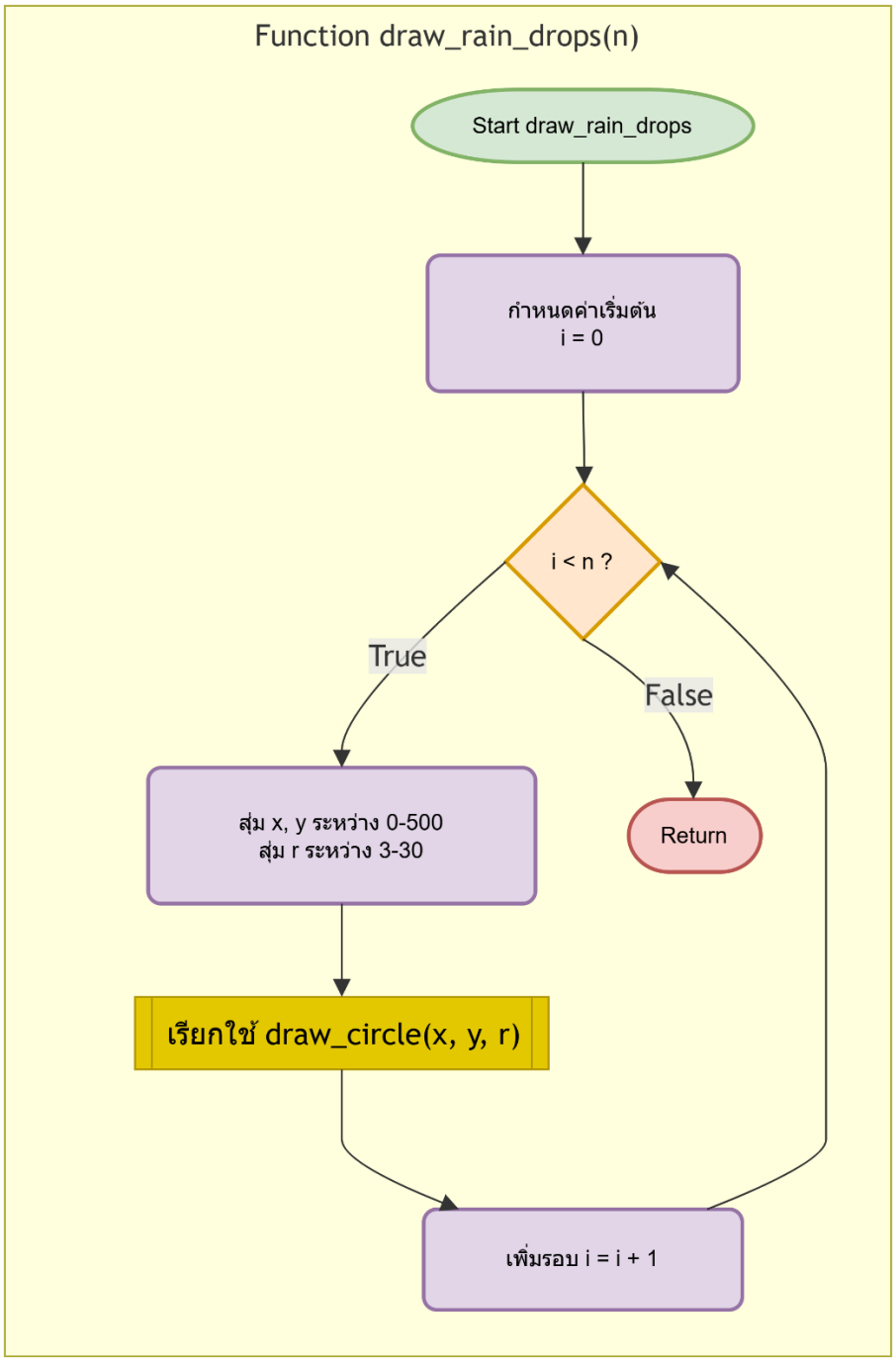
Flowchart แสดงการทำงาน

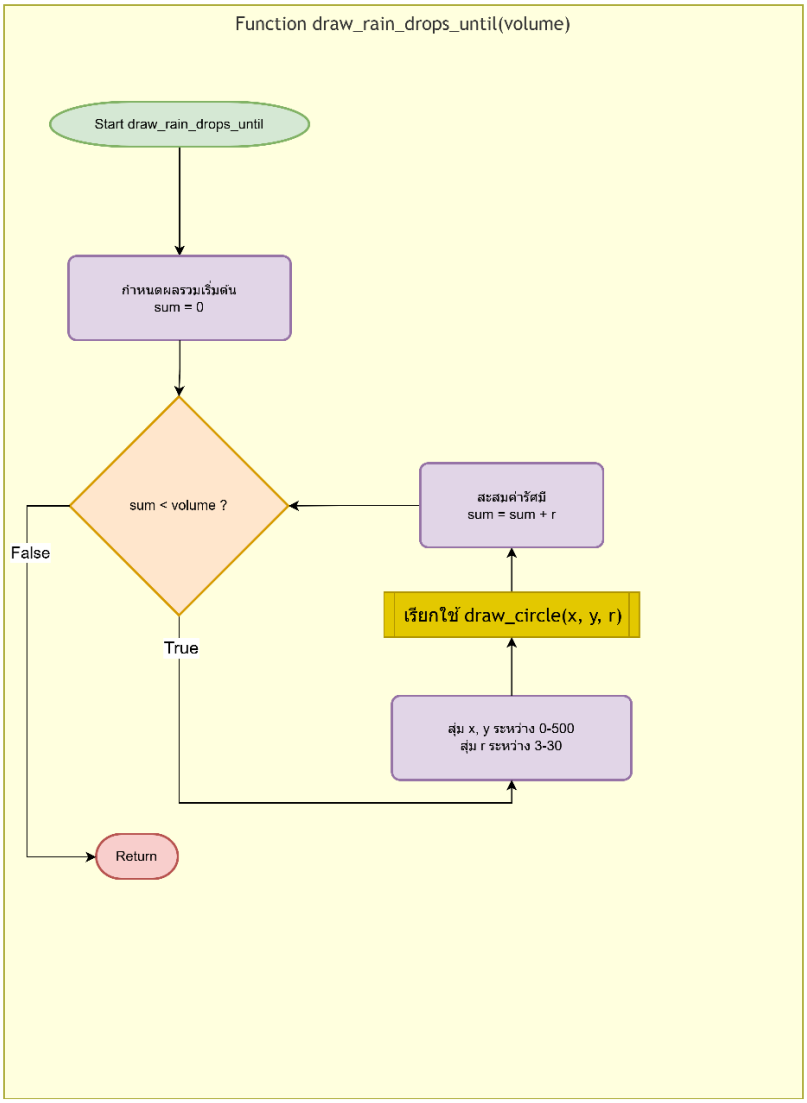


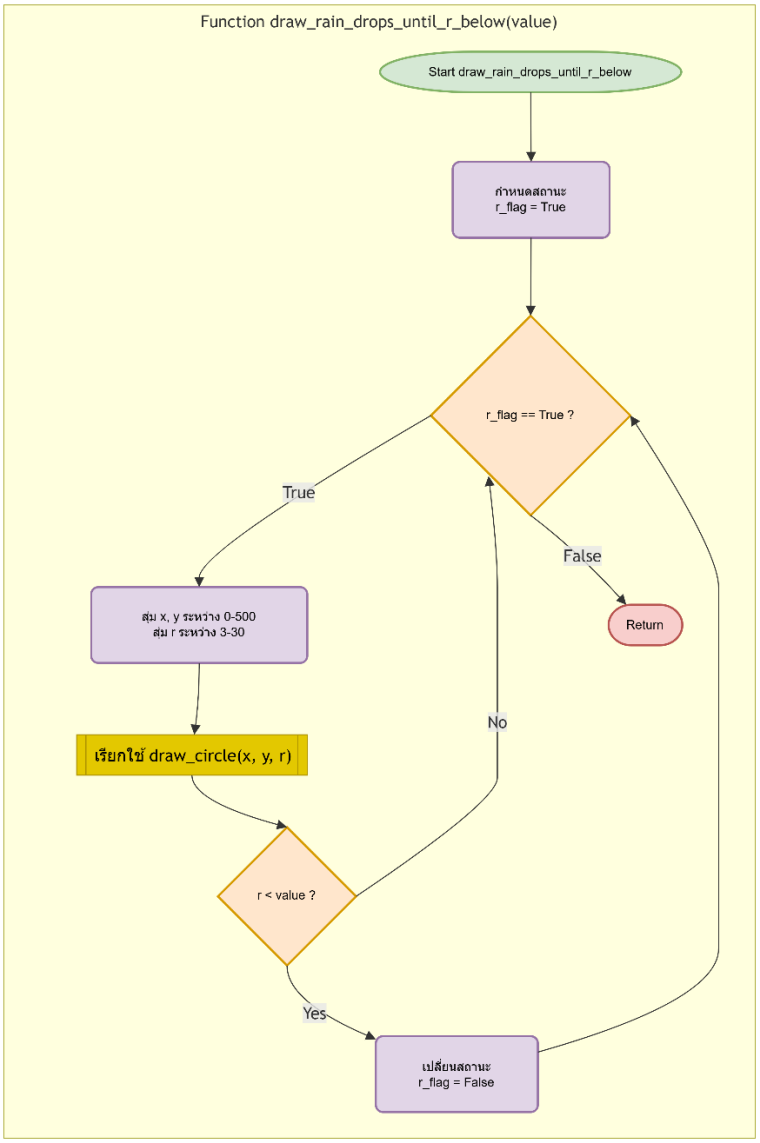



Function draw()
Start draw
ล้างพื้นหลัง
background 30, 40, 60
OPTION 1: กำหนดสีฟ้า
fill 100, 180, 255, 150
เรียกใช้ draw_rain_drops(50)
OPTION 2: กำหนดสีเขียว
fill 0, 255, 150, 100
เรียกใช้ draw_rain_drops_until(500)
OPTION 3: กำหนดสีแดง
fill 255, 100, 100, 100
เรียกใช้ draw_rain_drops_until_r_below(5)
หน่วงเวลา 100 ms
delay 100
Return


Function draw_circle(x, y, r)
Start draw_circle
วาดวงกลมเส้นผ่านศูนย์กลาง r*2
circle x, y, r * 2
Return


Function draw_rain_drops(n)
Start draw_rain_drops
กำหนดค่าเริ่มต้น
i = 0
i < n ?
True
False
สุ่ม x, y ระหว่าง 0-500
สุ่ม r ระหว่าง 3-30
Return
เรียกใช้ draw_circle(x, y, r)
เพิ่มรอบ i = i + 1


Function draw_rain_drops_until(volume)
Start draw_rain_drops_until
กำหนดผลรวมเริ่มต้น
sum = 0
sum < volume ?
False
True
สะสมค่ารัศมี
sum = sum + r
เรียกใช้ draw_circle(x, y, r)
สุ่ม x, y ระหว่าง 0-500
สุ่ม r ระหว่าง 3-30
Return


Function draw_rain_drops_until_r_below(value)
Start draw_rain_drops_until_r_below
กำหนดสถานะ
r_flag = True
r_flag == True ?
True
False
Return
สุ่ม x, y ระหว่าง 0-500
สุ่ม r ระหว่าง 3-30
เรียกใช้ draw_circle(x, y, r)
No
r < value ?
Yes
เปลี่ยนสถานะ
r_flag = False

Code และตัวอย่างInput/output

```
def setup():
    size(500, 500)
    background(30, 40, 60)

def draw():
    background(30, 40, 60)
    #OPTION : 1
    fill(100, 180, 255, 150)
    draw_rain_drops(50)

    #OPTION : 2
    fill(0, 255, 150, 100)
    draw_rain_drops_until(500)

    #OPTION : 3
    fill(255, 100, 100, 100)
    draw_rain_drops_until_r_below(5)
    delay(100)

def draw_circle(x, y, r):
    circle(x, y, r * 2)


#Slide page 47
def draw_rain_drops(n):
    i = 0
    while i < n:
        x = random(0, 500)
        y = random(0, 500)
        r = random(3, 30)
        draw_circle(x, y, r)
        i = i + 1

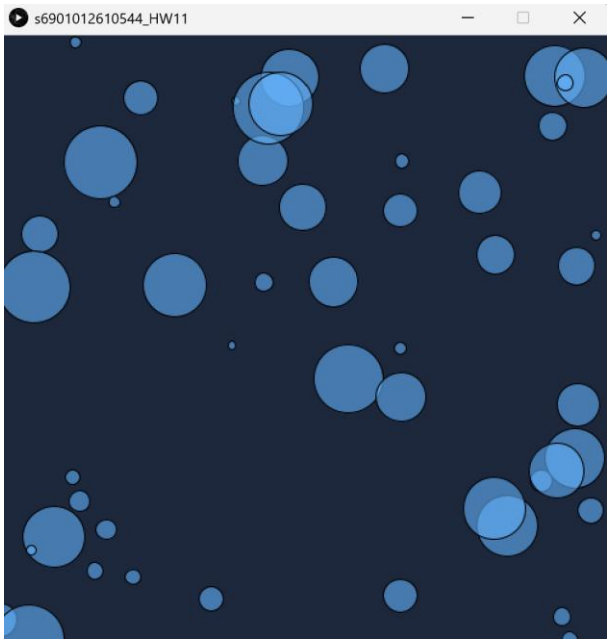

#Slide page 48
def draw_rain_drops_until(volume):
    sum = 0
    while sum < volume:
        x = random(0, 500)
        y = random(0, 500)
        r = random(3, 30)
        draw_circle(x, y, r)

        sum = sum + r

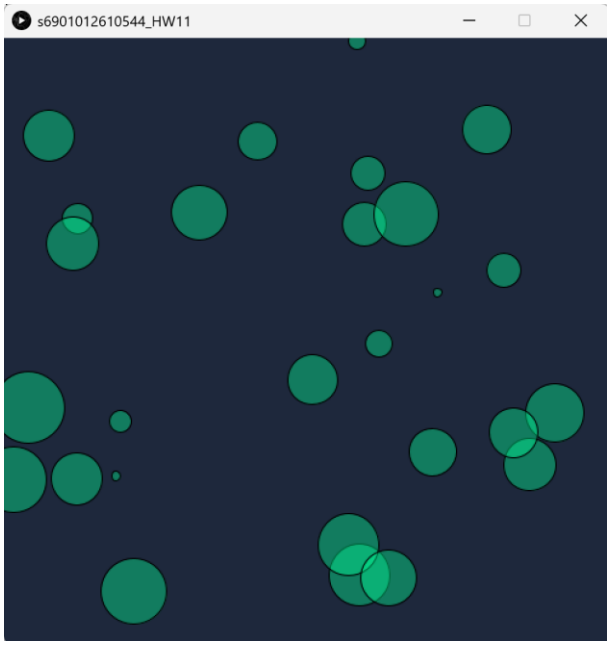

#Slide page 49
def draw_rain_drops_until_r_below(value):
    r_flag = True
    while r_flag:
        x = random(0, 500)
        y = random(0, 500)
        r = random(3, 30)
        draw_circle(x, y, r)

        if r < value:
            r_flag = False
```

ตัวอย่าง output ฟังก์ชัน draw_rain_drops(n)



ตัวอย่าง output ฟังก์ชัน draw_rain_drops_until(volume)



ตัวอย่าง output ฟังก์ชัน draw_rain_drops_until_r_below(value)

